

Styling XML – CSS, XSL, XPath & Transformations

XML Styling and Data Transformation Techniques

Presented by : Manal Asghar

Introduction to XML Styling

What is XML Styling?

- XML stores data but does not define how data should appear.
- Styling is used to display XML data in a readable format.
- XML can be styled using CSS and XSL.
- Styling improves presentation and user experience.
- XML styling is commonly used in web applications.
- Different styling methods provide different levels of control.

Main Styling Methods

- CSS (Cascading Style Sheets)
- XSL (Extensible Stylesheet Language)
- XSLT (XSL Transformations)

Purpose of XML Styling

- Improve readability
- Format XML content
- Transform XML data
- Display structured information clearly

CSS for XML

Using CSS with XML

- CSS can be applied directly to XML documents.
- CSS controls colors, fonts, spacing, and layout.
- XML elements are styled like HTML elements.
- CSS is simple and easy for basic styling.
- XML file links CSS using processing instruction.

Linking CSS with XML

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="style.css"?>
```

Advantages of CSS in XML

- Simple styling method
- Easy to learn
- Improves visual appearance
- Suitable for small XML projects

CSS Selectors in XML

CSS Selectors

- Selectors target XML elements for styling.
- CSS rules apply styles to matching elements.
- Selectors improve formatting control.

Example XML

```
<student>
  <name>Ali</name>
  <age>21</age>
</student>
```

CSS File

```
student {
  display: block;
  border: 1px solid black;
  padding: 10px;
}

name {
  color: blue;
  font-size: 20px;
}
```

Common Selectors

- Element selectors

- Class selectors
- ID selectors
- Universal selector

CSS Properties and Values

Common CSS Properties in XML

- CSS properties define the appearance of XML elements.
- Values determine how styles are applied.
- Multiple properties can be applied together.

Example

```
name {  
  color: red;  
  font-size: 24px;  
  font-weight: bold;  
}
```

Common CSS Properties

- color
- background-color
- font-size
- font-family
- margin
- padding
- border

Benefits

- Better presentation
- Easy formatting
- Improved readability

Cascading and Inheritance

Cascading in CSS

- Cascading means styles can come from multiple rules.
- CSS decides which style has higher priority.
- More specific selectors override general selectors.

Inheritance in CSS

- Child elements can inherit styles from parent elements.
- Inheritance reduces repeated styling code.
- Common inherited properties include font and color.

Example

```
student {  
  color: green;  
}
```

- Child elements inside `student` may inherit green color.

Advantages

- Cleaner stylesheet
- Reduced duplication
- Better style management

Introduction to XSL

What is XSL?

- XSL stands for Extensible Stylesheet Language.
- It is used to style and transform XML documents.
- XSL provides advanced formatting features.
- XSL is more powerful than CSS for XML.
- It supports XML transformation into HTML.

Components of XSL

- XSLT (Transformation)
- XPath (Navigation)
- Formatting Objects

Main Uses of XSL

- XML to HTML transformation
- XML formatting
- Data filtering
- Data presentation

Introduction to XSLT

What is XSLT?

- XSLT stands for XSL Transformations.
- It transforms XML into other formats.
- Common output formats include HTML and XML.
- XSLT uses templates and XPath expressions.
- XSLT is widely used in XML processing systems.

Features of XSLT

- Template-based processing
- XML transformation
- Dynamic content generation
- Supports loops and conditions

Advantages

- Powerful transformation system
- Flexible XML processing
- Reusable templates

Structure of XSLT

Basic Structure of XSLT

- XSLT documents start with `<xsl:stylesheet>`.
- Templates define transformation rules.
- `match` attribute selects XML elements.
- Output can be HTML, XML, or text.

Basic Example

```
<?xml version="1.0"?>

<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
    <html>
      <body>
        <h1>Student Data</h1>
      </body>
    </html>
```

```
</xsl:template>

</xsl:stylesheet>
```

Explanation

- `xsl:stylesheet` defines XSLT document
- `xsl:template` defines transformation rule
- `match="/"` selects root node

XPath Introduction

What is XPath?

- XPath is used to navigate XML documents.
- It selects nodes and elements from XML.
- XPath is commonly used with XSLT.
- XPath expressions help retrieve XML data.

XPath Can Select

- Elements
- Attributes
- Text nodes
- Specific values

Example

```
/student/name
```

Meaning

- Selects the `name` element inside `student`

XPath Expressions

Common XPath Expressions

Select Root Node

```
/
```

Select Specific Element

```
/student/name
```

Select All Elements

```
//name
```

Select Text Content

```
text()
```

Select Attributes

```
//@id
```

Advantages of XPath

- Fast XML navigation
- Accurate data selection
- Powerful query support

Using xsl:value-of

xsl:value-of

- `xsl:value-of` extracts and displays XML data.
- It selects values using XPath expressions.
- Frequently used in XSLT templates.

Example XML

```
<student>  
  <name>Ali</name>  
</student>
```

XSLT Example

```
<xsl:value-of select="student/name"/>
```

Output

Ali

Purpose

- Display XML data
- Extract element values
- Create dynamic output

Using `xsl:for-each`

`xsl:for-each` Loop

- `xsl:for-each` is used to loop through XML elements.
- It processes multiple nodes one by one.
- Commonly used for lists and tables.

Example

```
<xsl:for-each select="students/student">
  <p>
    <xsl:value-of select="name" />
  </p>
</xsl:for-each>
```

Explanation

- Loops through all `student` elements
- Displays each student name

Advantages

- Dynamic content generation
- Reusable processing
- Easy XML iteration

Using `xsl:if`

Conditional Processing with `xsl:if`

- `xsl:if` applies conditions in XSLT.
- It works like an if statement in programming.

- Used to display content conditionally.

Example

```
<xsl:if test="age > 18">  
  <p>Adult Student</p>  
</xsl:if>
```

Explanation

- Condition checks age value
- Content appears only if condition is true

Uses

- Conditional formatting
- Filtering XML data
- Dynamic output control

XML to HTML Transformation

Transforming XML into HTML

- XSLT can convert XML data into HTML pages.
- Transformation improves browser presentation.
- HTML output becomes easier for users to read.

Example Output

```
<h1>Student Record</h1>  
<p>Ali Khan</p>
```

Advantages

- User-friendly display
- Dynamic webpage generation
- Better data presentation

Practical Task — XML with CSS

Create XML File

student.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="style.css"?>

<student>
  <name>Ali Khan</name>
  <age>21</age>
</student>
```

Create CSS File

style.css

```
student {
  display: block;
  border: 2px solid black;
  padding: 10px;
}

name {
  color: blue;
  font-size: 22px;
}
```

Result

- XML data will appear with CSS styling.

Practical Task — XML with XSLT

student.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="student.xsl"?>

<student>
  <name>Ali Khan</name>
```

```
<age>21</age>
</student>
```

student.xsl

```
<?xml version="1.0"?>

<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
    <html>
      <body>
        <h1>Student Record</h1>
        <p><xsl:value-of select="student/name"/></p>
      </body>
    </html>
  </xsl:template>

</xsl:stylesheet>
```

Result

- XML data transforms into HTML format.

Advantages of XSLT and XPath

Benefits

XSLT Benefits

- Powerful XML transformation
- Dynamic content generation
- Reusable templates
- Flexible formatting

XPath Benefits

- Easy XML navigation
- Accurate node selection
- Fast data retrieval

Combined Advantages

- Professional XML processing

- Structured transformations
- Better data presentation

Conclusion

Summary

- CSS provides basic styling for XML.
- XSL and XSLT provide advanced transformation features.
- XPath helps navigate XML documents.
- `xsl:value-of` extracts XML values.
- `xsl:for-each` processes multiple elements.
- `xsl:if` applies conditional logic.
- XSLT transforms XML into HTML and other formats.

Final Note

CSS, XSLT, and XPath are important technologies for styling, transforming, and processing XML documents professionally.

Thank You

Questions & Discussion